11 Treasure Hunt Grid

AGES 8-12

MASCOT GOES HERE

KID INSTRUCTIONS

This is the big one. You've learned sequences, loops, conditionals, and debugging. Now use them ALL together to guide a robot named Bolt through a grid to find the treasure!

Navigate

Navigate — To find a path from one place to another. Programs that navigate are called pathfinders.
Example: Google Maps navigates you to a restaurant — it uses an algorithm to find the best path!

How It Works — Meet Bolt

Meet BOLT, your robot. Bolt understands just 3 commands:
MOVE FORWARD — move 1 square in the direction Bolt is facing **TURN LEFT** — rotate 90° to the left (stay in the same square) **TURN RIGHT** — rotate 90° to the right (stay in the same square)
Bolt always starts facing RIGHT unless the grid says otherwise.

Mini-example (3×3 grid)

```
    1     2     3
1 [   ] [   ] [ T ]
2 [   ] [   ] [   ]
3 [S▸] [   ] [   ]
```

Program to reach the treasure:

```
1. MOVE FORWARD
2. MOVE FORWARD
3. TURN LEFT
4. MOVE FORWARD
5. MOVE FORWARD
```

Bolt reaches the treasure!

Easy, right? Now let's make it harder — with WALLS in the way!

Basic Version

Get Bolt to the Treasure — Bolt starts at the top-left, facing RIGHT. The treasure is on the same row at the far right — but WALLS block the direct path! Write Bolt's program to reach the treasure without hitting any walls.

Grid 1

```
    1     2     3     4     5
1 [S▸] [   ] [###] [   ] [ T ]
2 [   ] [   ] [###] [   ] [   ]
3 [   ] [   ] [   ] [   ] [   ]
4 [   ] [   ] [   ] [   ] [   ]
5 [   ] [   ] [   ] [   ] [   ]
```

Bolt's Program:

1 _____
2 _____
3 _____
4 _____
5 _____
6 _____
7 _____
8 _____
9 _____
10 _____
11 _____
12 _____

How many commands did you use? _____

Challenge Version

PART A — Use a LOOP to Shorten Your Program

Bolt starts at the bottom-left, facing UP. The treasure is at the top-right. Walls block the direct path up — Bolt has to go around!
The trick: This path repeats the same move many times. Use a LOOP like "REPEAT MOVE 4 TIMES" to make your program shorter.

Grid 2 Start: bottom-left ▲, Treasure: top-right

```
    1     2     3     4     5
1 [   ] [   ] [   ] [   ] [ T ]
2 [   ] [   ] [   ] [   ] [   ]
3 [###] [###] [   ] [   ] [   ]
4 [   ] [   ] [   ] [   ] [   ]
5 [S▲] [   ] [   ] [   ] [   ]
```

Bolt's Program (with loops!):

1 _____
2 _____
3 _____
4 _____
5 _____
6 _____
7 _____
8 _____

How many commands?
Could you do it in FEWER commands? Try!

PART B — Debug Bolt's Program

A programmer wrote this program to guide Bolt from (5,1) facing UP to the treasure at (1,3). But there's a BUG — Bolt crashes into a wall!

Grid 3

```
    1     2     3     4     5
1 [###] [   ] [ T ] [   ] [   ]
2 [   ] [   ] [   ] [   ] [   ]
3 [   ] [   ] [   ] [   ] [   ]
4 [   ] [   ] [   ] [   ] [   ]
5 [S▲] [   ] [   ] [   ] [   ]
```

Buggy program:

```
1. MOVE FORWARD
2. MOVE FORWARD
3. MOVE FORWARD
4. MOVE FORWARD
5. TURN RIGHT
6. MOVE FORWARD
7. MOVE FORWARD
```

Finger-trace Bolt's path on the grid. Which step crashes into the wall?
What's the bug?

_____

_____

Write the corrected program:

1 _____
2 _____
3 _____
4 _____
5 _____
6 _____
7 _____
8 _____

Self-Check

- [ ] I got Bolt to the treasure in Grid 1 without hitting walls
- [ ] I used a LOOP to shorten my program in Grid 2
- [ ] I found and fixed the bug in Grid 3
- [ ] I can explain how SEQUENCE, LOOPS, and DEBUGGING all worked together

Reflection Box

Real navigation algorithms (like Google Maps) have to think about thousands of possible paths at once. How is what YOU just did similar? How is it different?

Bonus: Design Your Own Grid

Now flip it around! Design your OWN treasure hunt grid. Place a START, a TREASURE, and at least 3 walls. Then write the program that solves it.

Use a pencil — write "S" for start, "T" for treasure, and shade in 3+ walls. Don't forget to draw an arrow showing which direction S is facing!

Your Grid

```
    1   2   3   4   5
1 [   ] [   ] [   ] [   ] [   ]
2 [   ] [   ] [   ] [   ] [   ]
3 [   ] [   ] [   ] [   ] [   ]
4 [   ] [   ] [   ] [   ] [   ]
5 [   ] [   ] [   ] [   ] [   ]
```

Your Solution Program:

1 _____
2 _____
3 _____
4 _____
5 _____
6 _____
7 _____
8 _____

Trade your grid with a friend or family member! Did they solve it? How many commands did THEY use?

TEACHER / PARENT NOTE

This is the INTEGRATION activity. Up to now, each activity has taught ONE concept in isolation. This activity makes kids use them ALL TOGETHER — sequence (Activity 2) for the order of commands, loops (Activities 3-4) to shorten repeated moves, conditionals (Activities 5-6) when planning around walls, and debugging (Activity 7) when programs fail. This is also the first puzzle activity in the workbook — there's no single right answer, just paths that work or don't, which is genuine CS thinking. Discussion prompt: After your child finishes, sit together and walk through Bolt's program step by step on the grid — finger-trace Bolt's path to make the program execution visible. This is how real programmers debug.